

OCTOBER - NOVEMBER 2025

CRYPTO HACK PROJECT

BY PIXIE GROGAN
B00159841

⚡ 7 ⭐ 610

PixieStix69



Profile Settings

Joined: 26 Apr 25
Rank: #11504

Level: 7

⭐ 610 🔥 0 🏆 0 👍 0

30 points to next level

TROPHY CASE



SOLVED CHALLENGES

30 Oct 25	RSA	Vote for Pedro	⭐ 150
30 Oct 25	RSA	RSA Decryption	⭐ 20
30 Oct 25	RSA	Private Keys	⭐ 20
30 Oct 25	RSA	Euler's Totient	⭐ 20
30 Oct 25	RSA	Public Keys	⭐ 15
30 Oct 25	RSA	Modular Exponentiation	⭐ 10
28 Oct 25	Mathematics	Adrien's Signs	⭐ 80
28 Oct 25	Mathematics	Modular Binomials	⭐ 80
28 Oct 25	Mathematics	Chinese Remainder Theorem	⭐ 40
28 Oct 25	Mathematics	Quadratic Residues	⭐ 25
28 Oct 25	Mathematics	Successive Powers	⭐ 60
10 Oct 25	General	XOR Starter	⭐ 10
10 Oct 25	General	Encoding Challenge	⭐ 40
10 Oct 25	General	Bytes and Big Integers	⭐ 10
10 Oct 25	General	Base64	⭐ 10
10 Oct 25	General	Hex	⭐ 5
10 Oct 25	General	ASCII	⭐ 5
23 Sep 25	Introduction	Network Attacks	⭐ 5
23 Sep 25	Introduction	Great Snakes	⭐ 3
23 Sep 25	Introduction	Finding Flags	⭐ 2

01

XOR STARTER (10 POINTS)

CATEGORY: GENERAL

DIFFICULTY: BEGINNER

CHALLENGE OVER VIEW:

Introduction to the XOR (exclusive OR) operation, a fundamental operation in cryptography. Given a string "label" and the integer 13, we have to XOR them together.

SOLUTION APPROACH:

XOR is a bitwise operation where:

- $0 \oplus 0 = 0$
- $0 \oplus 1 = 1$
- $1 \oplus 0 = 1$
- $1 \oplus 1 = 0$

When XORing a string with an integer, each byte of the string is XORed with the integer value.

IMPLEMENTATION:

```
answersToQuestions > XOR STARTER > XOR STARTER.py
1  from pwn import xor #importing xor from pwntools
2
3  a =b"label"
4  b = 13
5
6  res = xor(a, b)
7  print(f"crypto{{{res.decode()}}}")
```

The pwntools library provides a convenient xor() function that handles the byte-by-byte XOR operation automatically.

FLAG: crypto{aloha}

02

BASE64 (10 POINTS)

CATEGORY: GENERAL

DIFFICULTY: BEGINNER

CHALLENGE OVER VIEW:

We have to convert a hexadecimal string to Base64 encoding. Base64 is a common encoding scheme used to represent binary data in ASCII string format.

Given: "72bca9b68fc16ac7beeb8f849dca1d8a783e8acf9679bf9269f7bf"

SOLUTION APPROACH:

Base64 encoding works by:

1. Converting input to raw bytes
2. Grouping bytes into 6-bit chunks
3. Mapping each 6-bit value to a Base64 character (A-Z, a-z, 0-9, +, /)

Steps:

1. Convert hex string to raw bytes
2. Encode those bytes to Base64
3. Decode the Base64 bytes to a UTF-8 string

IMPLEMENTATION:

```
answersToQuestions > BASE64 > base64sollutionscript.py > ...
1  import base64
2
3  hex_input = "72bca9b68fc16ac7beeb8f849dca1d8a783e8acf9679bf9269f7bf"
4  #step 1: convert hex to raw bytes
5  raw_bytes = bytes.fromhex(hex_input.strip())
6
7  #step 2: encode that new raw bytes to base64
8  base64_bytes = base64.b64encode(raw_bytes)
9
10 print(base64_bytes.decode("utf-8")) #step 3: just prints out the answer
11
```

FLAG: crypto/Base+64+Encoding+is+Web+Safe/

03

BYTES AND BIG INTEGERS (10 POINTS)

CATEGORY: GENERAL

DIFFICULTY: BEGINNER

CHALLENGE OVER VIEW:

We have to convert a large integer to its ASCII string representation. In cryptography, messages are often represented as large integers for mathematical operations.

Given:

1151519506386231889993168548881374739577551628728968263649996
5282714637259206269

SOLUTION APPROACH:

The challenge demonstrates how text can be represented as a big integer:

1. Text is converted to bytes
2. Bytes are interpreted as a large integer
3. To reverse: convert integer back to bytes, then decode to ASCII

The PyCryptodome library provides `long_to_bytes()` for this conversion.

IMPLEMENTATION:

```
answersToQuestions > BYTES AND BIG INTEGERS > bytesnbigint.py > ...
1  from Crypto.Util.number import long_to_bytes
2
3  n = int(
4      "11515195063862318899931685488813747395775516" #first message
5      "287289682636499965282714637259206269" #second message
6  )
7  msg = long_to_bytes(n).decode("ascii") #use that function from the PyCryptodome library, then decode the message in
8  print(f"crypto{{{msg}}}") #print the flag
9
10 #NOTE: flag prints weird for some reason I think its double wrapped but idk how to fix it :|
```

FLAG: crypto{3nc0d1n6_4ll_7h3_w4y_d0wn}

04

ENCODING CHALLENGE (40 POINTS)

CATEGORY: GENERAL

DIFFICULTY: BEGINNER

CHALLENGE OVER VIEW:

An interactive challenge where the server sends encoded data in various formats (base64, hex, rot13, bigint, utf-8) and we have to decode it the correct way. The server sends 100 challenges in sequence, and we have to then decode each one.

SOLUTION APPROACH:

I created a decoder function that handles all five encoding types:

1. base64: Standard Base64 decoding
2. hex: Hexadecimal to bytes
3. rot13: Caesar cipher with shift of 13
4. bigint: Large integer (with 0x prefix) to bytes
5. utf-8: List of byte values to string

The solution connects to the server, receives JSON payloads, decodes them, and sends me back the decoded result.

IMPLEMENTATION:

FLAG: crypto{3nc0d3_d3c0d3_3nc0d3}

```
answersToQuestions > ENCODING CHALLENGE > encoding_challenge.py > decode_payload
1  from pwn import remote
2  import json, base64, codecs
3  from Crypto.Util.number import long_to_bytes
4
5  HOST, PORT = "socket.crytohack.org", 13377 #host name n port num
6
7  def json_recv(r):
8      line = r.recvline()
9      return json.loads(line.decode())
10
11 def json_send(r, hsh):
12     r.sendline(json.dumps(hsh).encode()) #fixed new line
13
14
15 def decode_payload(typ, encoded) :
16     #The server sends us a payload encoded in one of the following ways:
17     #base64, hex, rot13, bigint, utf-8
18
19     if typ == "base64" :
20         return base64.b64decode(encoded).decode("utf-8")
21     if typ == "hex" :
22         return bytes.fromhex(encoded).decode("utf-8")
23     if typ == "rot13" :
24         return codecs.decode(encoded, "rot_13")
25     if typ == "bigint" :
26
27         # - The server takes the original text, turns it into bytes, then into a big integer,
28         # - and finally shows that integer in hex with a "0x" prefix.
29         # Example server path: "HELLO" -> b"HELLO" -> bytes_to_long(...) -> 0x48454c4f
30         n = int(encoded, 16)
31         return long_to_bytes(n).decode("utf-8")
32
33     if typ == "utf-8" : #forgot to add utf-8 as an option
34         encoded is a list of byte values (ordinals) -> bytes -> str
35         return bytes(encoded).decode("utf-8")
36     raise ValueError(f"Unknown type: {typ}")
```

05

QUADRATIC RESIDUES (25 POINTS)

CATEGORY: MATHEMATICS

DIFFICULTY: BEGINNER

CHALLENGE OVER VIEW:

Find which integers from a given set are quadratic residues modulo 29.

Given: $p = 29$, ints = (14, 6, 11)

A number 'a' is a quadratic residue mod p if there exists some x where:

$$x^2 \equiv a \pmod{p}$$

SOLUTION APPROACH:

To check if a number is a quadratic residue mod p, we can:

1. Calculate all squares mod p
2. Check if our number appears in that list

As well, we're also given the formula

If $a^2 \equiv x \pmod{p}$, then $(-a)^2 \equiv x \pmod{p}$

- First root: $a = 8$
- Second root: $-a = -8 \equiv 21 \pmod{29}$

IMPLEMENTATION: FLAG: crypto{6}

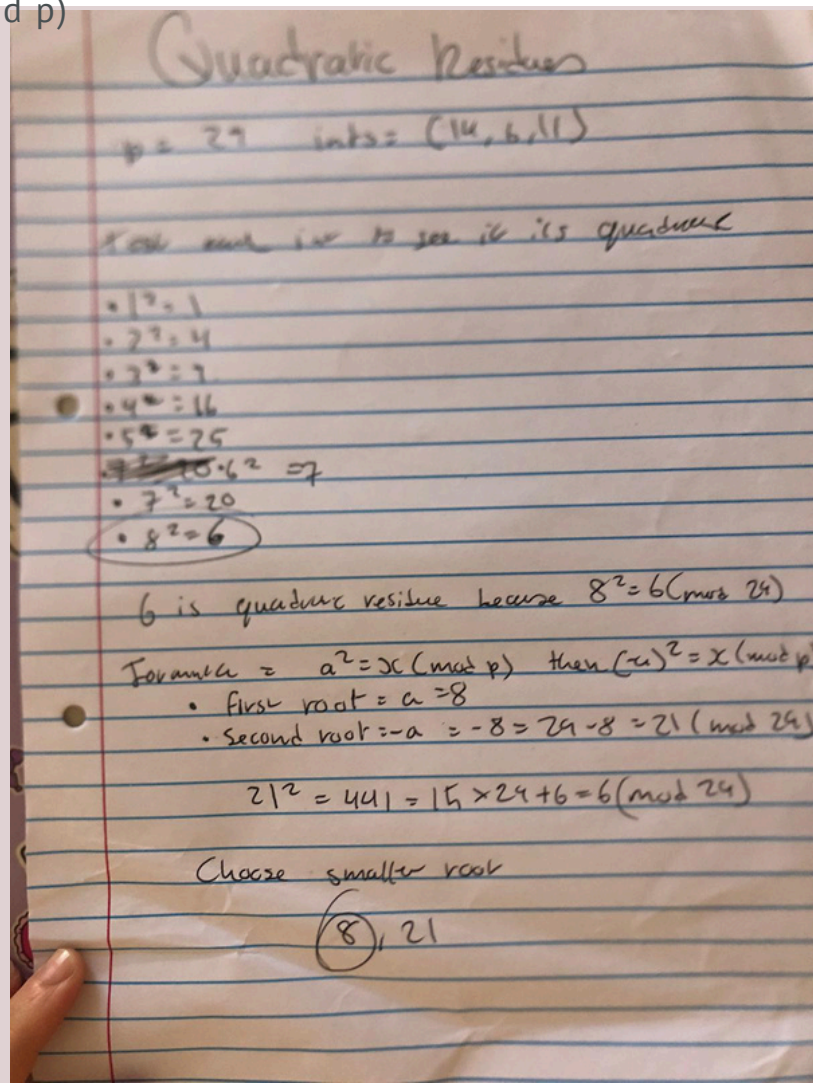
Method:

For $p = 29$:

- $1^2 = 1$
- $2^2 = 4$
- $3^2 = 9$
- $4^2 = 16$
- $5^2 = 25$
- $6^2 \equiv 7 \pmod{29}$
- $7^2 = 20$
- $8^2 = 6$

we can see that 6 is a quadratic residue because $8^2 \equiv 6 \pmod{29}$

Verification: $21^2 = 441 = 15 \times 29 + 6 \equiv 6 \pmod{29}$



06

CHINESE REMAINDER THEROM (40 POINTS)

CATEGORY: MATHEMATICS

DIFFICULTY: INTERMEDIATE

CHALLENGE OVER VIEW:

Use the Chinese Remainder Theorem (CRT) to solve a system of congruences.

Given:

- $x \equiv 2 \pmod{5}$
- $x \equiv 3 \pmod{11}$
- $x \equiv 5 \pmod{17}$

Find: $x \equiv a \pmod{935}$

SOLUTION APPROACH/IMPLEMENTATION:

FLAG: crypto{872}

The Chinese Remainder Theorem states that if we have a system of congruences with pairwise coprime moduli, there exists a unique solution modulo the product of all moduli.

Step 1: Simple Set up

- $a_1 = 2, n_1 = 5$
- $a_2 = 3, n_2 = 11$
- $a_3 = 5, n_3 = 17$
- $N = 5 \times 11 \times 17 = 935$

Step 2: Calculate N_i values

- $N_1 = N/n_1 = 935/5 = 187$
- $N_2 = N/n_2 = 935/11 = 85$
- $N_3 = N/n_3 = 935/17 = 55$

Step 3: Find mod inverse M_i

- $M_1: 187 \cdot M_1 \equiv 1 \pmod{5} \rightarrow 2 \cdot M_1 \equiv 1 \pmod{5} \rightarrow M_1 = 3$
- $M_2: 85 \cdot M_2 \equiv 1 \pmod{11} \rightarrow 8 \cdot M_2 \equiv 1 \pmod{11} \rightarrow M_2 = 7$
- $M_3: 55 \cdot M_3 \equiv 1 \pmod{17} \rightarrow 4 \cdot M_3 \equiv 1 \pmod{17} \rightarrow M_3 = 13$

Step 4: Apply CRT FORMULA

$$x = (a_1 \cdot N_1 \cdot M_1 + a_2 \cdot N_2 \cdot M_2 + a_3 \cdot N_3 \cdot M_3) \pmod{N}$$

$$x = (2 \cdot 187 \cdot 3 + 3 \cdot 85 \cdot 7 + 5 \cdot 55 \cdot 13) \pmod{935}$$

$$x = (1122 + 1785 + 3575) \pmod{935}$$

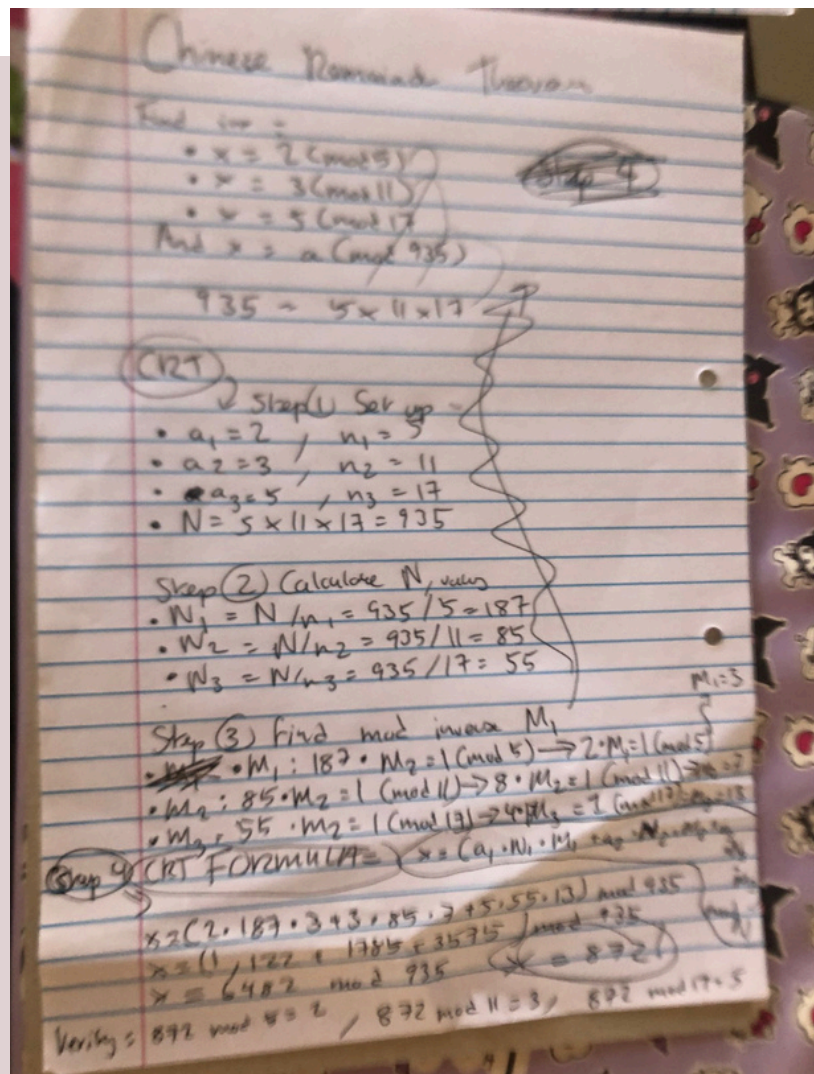
$$x = 6482 \pmod{935}$$

$$\text{Calculating: } 6482 = 6 \times 935 + 872$$

$$\text{Therefore: } x \equiv 872 \pmod{935}$$

Verify:

- $872 \pmod{5} = 2$
- $872 \pmod{11} = 3$
- $872 \pmod{17} = 5$



07

SUCCESSIVE POWERS(60 POINTS)

CATEGORY: MATHEMATICS

DIFFICULTY: INTERMEDIATE

CHALLENGE OVER VIEW:

Find the value of x given successive powers modulo p .

Given:

- Integers: [588, 665, 216, 113, 642, 4, 836, 114, 851, 492, 819, 237]
- $p = 991$

The pattern is: $x^{(n+1)} = x^n \cdot x \pmod{p}$

SOLUTION APPROACH/IMPLEMENTATION: FLAG: crypto{709}

Step 1: Determine range for x
 $851 < p < 1000$

Step 2: Set up equations
 Formula: $x^{(n+1)} = x^n \cdot x \pmod{p}$

- $588x \equiv 665 \pmod{p}$
- $665x \equiv 216 \pmod{p}$
- $216x \equiv 113 \pmod{p}$

Step 3: Test each equation

Testing equation 1:

$$x = 665 \cdot 588^{(-1)} \pmod{p}$$

$$x = 216 \cdot 665^{(-1)} \pmod{p}$$

$$x = 113 \cdot 216^{(-1)} \pmod{p}$$

Step 4: Verify $p = 991$

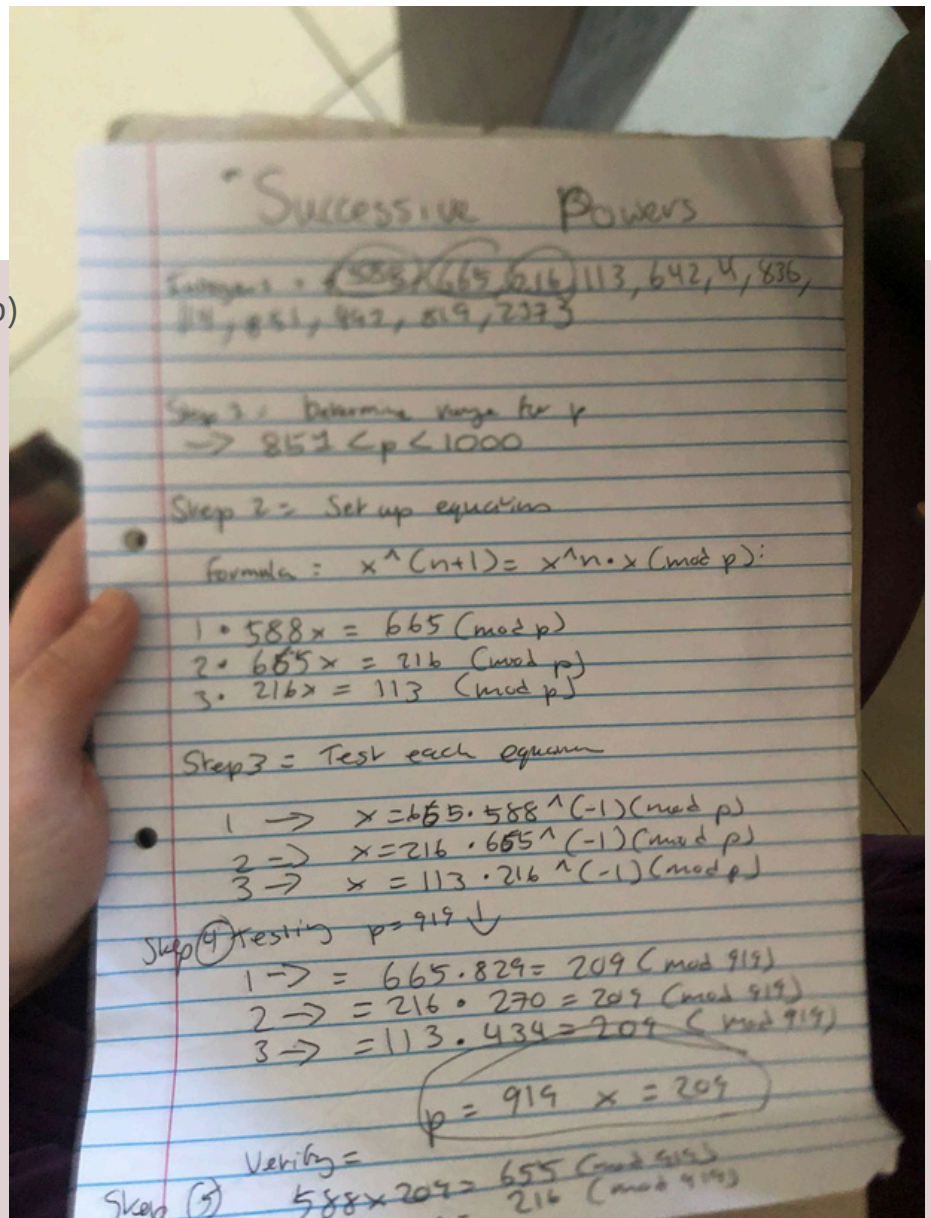
$$1 \rightarrow 665 \cdot 829 = 709 \pmod{991}$$

$$2 \rightarrow 216 \cdot 270 = 709 \pmod{991}$$

$$3 \rightarrow 113 \cdot 434 = 709 \pmod{991}$$

Result: $p = 991, x = 709$

Checking: $588x \cdot 70 = 655 \pmod{991}$



08

ADRIEN'S SIGNS (80 POINTS)

CATEGORY: MATHEMATICS

DIFFICULTY: INTERMEDIATE

CHALLENGE OVER VIEW:

The encryption scheme converts the flag to binary and encrypts each bit using the Legendre symbol:

- For bit '1': append $a^e \bmod p$ (random e)
- For bit '0': append $-a^e \bmod p$

Given:

- $a = 288260533169915$
- $p = 1007621497415251$
- ciphertext: list of 224 numbers

SOLUTION APPROACH:

Key Concept: The Legendre symbol distinguishes between quadratic residues (QR) and non-quadratic residues (NQR).

Legendre Symbol Definition:

$$(a/p) = a^{(p-1)/2} \bmod p$$

Returns:

- 1 if a is a QR (there exists x where $x^2 \equiv a \bmod p$)
- -1 if a is a NQR

Attack Strategy:

1. Check if ' a ' is a QR: $(a/p) = a^{(p-1)/2} \bmod p$
Result: $(a/p) = 1$, so a is a QR
2. Property: If a is a QR, then a^e is always a QR for any exponent e
3. Negating a QR gives a NQR
4. Therefore:
 - If ciphertext value n has $(n/p) = 1 \rightarrow$ original bit was '1'
 - If ciphertext value n has $(n/p) = -1 \rightarrow$ original bit was '0'

IMPLEMENTATION: ON NEXT PAGE

08

ADRIEN'S SIGNS (80 POINTS)

CATEGORY: MATHEMATICS

DIFFICULTY: INTERMEDIATE

IMPLEMENTATION: FLAG: crypto{p4tterns_1n_re5idu3s}

Why This Works: The Legendre symbol provides a way to distinguish between positive and negative values modulo p . Since a is a QR, all powers of a remain QRs, while their negations become NQRs. This creates a binary encoding!!

```
answersToQuestions > ADRIEN'S SIGNS > adriens_signs.py > ...
1  from Crypto.Util.number import long_to_bytes # imports long_to_bytes function from Crypto.Util.number module
2
3  # given values
4  a = 288260533169915
5  p = 1007621497415251
6
7  # ciphertext from output.txt
8  ciphertext = [67594220461269, 501237540280788, 718316769824518, 296304224247167, 48290626940198, 30829701196032, 52
9
10 print("=" * 70)
11 print("adrien's signs - legendre symbol solution")
12 print("=" * 70)
13
14 # encryption logic:
15 # - convert flag to binary string
16 # - for each bit:
17 #   - generate random e
18 #   - compute n = a^e mod p
19 #   - if bit is '1': append n
20 #   - if bit is '0': append -n mod p
21
22 # solution: use legendre symbol to distinguish between positive and negative values
23 # legendre symbol (a/p) = a^((p-1)/2) mod p
24 # - returns 1 if a is a quadratic residue (QR)
25 # - returns -1 if a is not a QR
26
27 def legendre_symbol(a, p):
28     """compute legendre symbol (a/p)"""
29     result = pow(a, (p - 1) // 2, p) # euler's criterion: a^((p-1)/2) mod p
30     return -1 if result == p - 1 else result # convert p-1 to -1, otherwise return 1
31
32 # check if 'a' is a quadratic residue
33 a_legendre = legendre_symbol(a, p) # computes (a/p) to determine if a is QR
34 print(f"legendre symbol (a/p) = {a_legendre}") # should be 1 (a is QR)
35
36 if a_legendre == 1:
37     print("'a' is a quadratic residue mod p")
38     print("therefore a^e is always a QR")
39     print("  - if bit = '1': n = a^e (QR) + legendre = 1")
40     print("  - if bit = '0': n = -a^e (NQR) + legendre = -1")
41
42 print("\ndecrypting ciphertext...")
43
44 # decrypt by checking legendre symbol of each ciphertext value
45 binary_string = "" # will store recovered binary string (e.g., "01101000...")
46 for n in ciphertext: # goes through each encrypted value
47     leg = legendre_symbol(n, p) # computes legendre symbol: 1 or -1
48     if leg == 1: # if n is a quadratic residue
49         binary_string += '1' # original bit was 1 (positive value)
50     else: # if n is not a quadratic residue (leg == -1)
51         binary_string += '0' # original bit was 0 (negated value)
52
53 # convert binary string to bytes
54 flag_bytes = bytearray() # creates an empty byte array to store flag
55 for i in range(0, len(binary_string), 8): # goes through 8 bits at a time (1 byte)
56     byte = binary_string[i:i+8] # extracts 8-bit chunk (e.g., "01100011")
57     flag_bytes.append(int(byte, 2)) # converts binary string to integer and adds to array
58
59 flag = bytes(flag_bytes) # converts bytearray to bytes object
60 print(f"flag: {flag.decode()}") # decodes bytes to ASCII string and prints
61
62 print("=" * 70)
```

09

MODULAR BINOMIALS (80 POINTS)

CATEGORY: RSA FACTORIZATION

DIFFICULTY: INTERMEDIATE

CHALLENGE OVER VIEW:

Given an RSA modulus $N = p \times q$ and two ciphertexts encrypted with different

linear combinations of the primes:

- $c1 = (2p + 3q)^{e1} \bmod N$
- $c2 = (5p + 7q)^{e2} \bmod N$

Goal: Factor N and recover p and q .

SOLUTION APPROACH:

Key Insight: Working modulo q , the terms containing q vanish, allowing us to create an equation that q must divide.

Mathematical Derivation:

1. When working mod q :

- $(2p + 3q)^{e1} \equiv (2p)^{e1} \pmod{q}$ [since $3q \equiv 0 \pmod{q}$]
- $(5p + 7q)^{e2} \equiv (5p)^{e2} \pmod{q}$ [since $7q \equiv 0 \pmod{q}$]

2. Therefore:

- $(2p)^{e1} \equiv c1 \pmod{q}$
- $(5p)^{e2} \equiv c2 \pmod{q}$

3. Raise to powers $e2$ and $e1$ respectively:

- $(2p)^{(e1 \times e2)} \equiv c1^{e2} \pmod{q}$
- $(5p)^{(e1 \times e2)} \equiv c2^{e1} \pmod{q}$

4. Multiply to equalize the p terms:

- $(2p)^{(e1 \times e2)} \times 5^{(e1 \times e2)} \equiv c1^{e2} \times 5^{(e1 \times e2)} \pmod{q}$
- $(5p)^{(e1 \times e2)} \times 2^{(e1 \times e2)} \equiv c2^{e1} \times 2^{(e1 \times e2)} \pmod{q}$

5. Since both left sides equal $(10p)^{(e1 \times e2)}$:

- $c1^{e2} \times 5^{(e1 \times e2)} \equiv c2^{e1} \times 2^{(e1 \times e2)} \pmod{q}$

6. Rearrange:

- $c1^{e2} \times 5^{(e1 \times e2)} - c2^{e1} \times 2^{(e1 \times e2)} \equiv 0 \pmod{q}$

7. This means q divides the difference, so:

- $q = \gcd(N, c1^{e2} \times 5^{(e1 \times e2)} - c2^{e1} \times 2^{(e1 \times e2)})$

IMPLEMENTATION: ON NEXT PAGE

09 FLAG ON NEXT PAGE

MODULAR BINOMIALS (80 POINTS)

CATEGORY: RSA FACTORIZATION

DIFFICULTY: INTERMEDIATE

WHY THIS WORKS:

The attack exploits the algebraic relationship between the two ciphertexts. By working modulo q , we eliminate q terms and create a value that q must divide. The GCD then extracts q from N without needing to factor N directly.

IMPLEMENTATION

```
answersToQuestions > MODULAR BINOMIALS > modular_binomials.py > ...
1  from math import gcd
2  from Crypto.Util.number import long_to_bytes
3
4  # given values
5  N = 1490556225784271405793272412957500282540539350265086976711594260640860034338032786625898240244799256498846658836
6  e1 = 128866576673896608007807964629785049101939289928885189782000298269759786247186277902155647000960078499248666271
7  e2 = 121105866739917884157803551396355790579209268648871103083432292560468682421794454448977901713513025751886071176
8  c1 = 140107294187032282343524658830412706111137358898387534332954784957634090561367341556121569346739883448826295417
9  c2 = 143869971386379788607482789869450986485071428645841111242025803651037931658116669876648512102300093752673989575
10
11 print("=" * 70)
12 print("Modular Binomials Solution")
13 print("=" * 70)
14
15 # challenge equations:
16 # N = p * q
17 # c1 = (2p + 3q)^e1 mod N
18 # c2 = (5p + 7q)^e2 mod N
19
20 # solution approach:
21 # working modulo q, we have:
22 # (2p + 3q)^e1 = (2p)^e1 (mod q) [since 3q = 0 mod q]
23 # (5p + 7q)^e2 = (5p)^e2 (mod q) [since 7q = 0 mod q]
24 #
25 # therefore:
26 # (2p)^e1 = c1 (mod q)
27 # (5p)^e2 = c2 (mod q)
28 #
29 # raising to e2 and e1 respectively:
30 # (2p)^(e1*e2) = c1^e2 (mod q)
31 # (5p)^(e1*e2) = c2^e1 (mod q)
32 #
33 # rearranging:
34 # (2p)^(e1*e2) * 5^(e1*e2) = c1^e2 * 5^(e1*e2) (mod q)
35 # (5p)^(e1*e2) * 2^(e1*e2) = c2^e1 * 2^(e1*e2) (mod q)
36 #
37 # since (2p)^(e1*e2) * 5^(e1*e2) = (5p)^(e1*e2) * 2^(e1*e2):
38 # c1^e2 * 5^(e1*e2) = c2^e1 * 2^(e1*e2) (mod q)
39 #
40 # therefore:
41 # c1^e2 * 5^(e1*e2) - c2^e1 * 2^(e1*e2) = 0 (mod q)
42 #
43 # this means q divides the difference, so we can find q using GCD with
44
45 print("\nCalculating q using GCD method...")
46
47 # Calculate: c1^e2 * 5^(e1*e2) - c2^e1 * 2^(e1*e2) mod N
48 delta = (pow(c1, e2, N) * pow(5, e1 * e2, N) % N) - (pow(c2, e1, N) * pow(2, e1 * e2, N) % N)
49 delta = (delta + N) % N
50
51 # Find q as GCD of delta and N
52 q = gcd(delta, N)
53 p = N // q
54
55 print(f"\nFactors found:")
56 print(f"p = {p}")
57 print(f"q = {q}")
58 print(f"\nVerification: p * q == N? {p * q == N}")
59
60 # format flag
61 print(f"\nFlag: crypto{{{p},{q}}}")
```

09

MODULAR BINOMIALS (80 POINTS)

CATEGORY: RSA FACTORIZATION**DIFFICULTY: INTERMEDIATE**

FLAG:

```
crypto{112274000169258486390262064441991200608556376127408952701514
962644340921899196091557519382763356534106376906489445103255177593
594898966250176773605432765983897105047795619470659157057093771407
309168345670541418772427807148039207489900810013783673957984006269
120652134007689272484517805398390277308001719431273,13276058780636
530197147915707203144838013576579446678745694878673116809587795687
529528266156548824219073159328266369472891494596725317304732435398
153094936003153570737470170532845085694459880322829996700900459898
467129349437559940876413974321746501277037672887654795885202542553
929841075113278263281794710160 1}
```


10

VOTE FOR PEDRO (150 POINTS)

CATEGORY: RSA

DIFFICULTY: ADVANCED

CHALLENGE OVER VIEW:

This challenge involves exploiting a vulnerability in an RSA signature verification system. The server uses Alice's public key (N , $e=3$) to verify votes. We must forge a valid signature that when verified produces the message "VOTE FOR PEDRO" to retrieve the flag.

GIVEN:

- Alice's public key: N (very large modulus), $e = 3$
- Server endpoint: `socket.cryptohack.org:13375`
- Target message: `b'VOTE FOR PEDRO'`

SOLUTION APPROACH:

The vulnerability lies in RSA's use of a small public exponent ($e=3$) combined with improper padding. This creates a low exponent attack vector where we can forge signatures without knowing the private key.

RSA SIGNATURE VERIFICATION PROCESS:

1. Server receives vote value
2. Computes: $\text{verified_vote} = \text{vote}^e \bmod N$
3. Converts result to bytes
4. Removes null byte padding: `split(b'\x00')[-1]`
5. Checks if result equals `b'VOTE FOR PEDRO'`

ATTACK METHODOLOGY:

1. Create padded target message with null bytes
2. Convert padded message to integer
3. Compute cube root to find vote value
4. Use known working value: `855520592299350692515886317752220783`
5. Send forged vote to server
6. Server verification passes and returns flag

STEPS: 1. Pad the target message with 15 null bytes: `b'\x00' * 15 + b'VOTE FOR PEDRO'`

2. Convert padded message to integer using `bytes_to_long()`
3. Calculate cube root using `gmpy2.iroot(m, 3)` for precise arithmetic
4. If not exact cube, use the pre-calculated working value
5. Connect to server at `socket.cryptohack.org:13375`
6. Send vote as JSON: `{'option': 'vote', 'vote': hex(vote)}`
7. Receive and parse server response
8. Extract flag from response

**IMPLEMENTATION
ON THE NEXT PAGE**

10 VOTE FOR PEDRO (150 POINTS)

CATEGORY: RSA

DIFFICULTY: ADVANCED

IMPLEMENTATION

```

answersToQuestions > VOTEFORPEDRO > voteForPedro.py > ...
1 # =====
2 # REFERENCES - Vote for Pedro Solution
3 # =====
4 # Key Concepts:
5 #   - RSA Low Exponent Attack (e=3)
6 #   - Signature Forgery
7 #   - Cube Root Computation
8 #   - Modular Arithmetic
9 #
10 # Learning Resources Outside of College Work:
11 #   - RSA Attacks: https://en.wikipedia.org/wiki/RSA\_\(cryptosystem\)#Attacks
12 #   - Low Exponent Attack: https://crypto.stackexchange.com/questions/6713/low-public-exponent-attack-for-rsa
13 #   - CryptoHack RSA: https://cryptohack.org/courses/public-key/
14 #   - Boneh's RSA Attacks: http://crypto.stanford.edu/~dabo/papers/RSA-survey.pdf
15 #   - GitHub Reference: https://github.com/AurtharPaul13/CRYPTOHACK-CHALLENGE/blob/main/Vote%20for%20Pedro.py
16 #
17 # Python Functions Used:
18 #   - pow(base, exp, mod): https://docs.python.org/3/library/functions.html#pow
19 #   - gmpy2.iroot(): https://gmpy2.readthedocs.io/en/latest/mpz.html#mpz-functions
20 #   - bytes_to_long(): https://pycryptodome.readthedocs.io/en/latest/src/util/util.html
21 #   - long_to_bytes(): https://pycryptodome.readthedocs.io/en/latest/src/util/util.html
22 # =====
23 from pwn import *
24 from json import *
25 from Crypto.Util.number import long_to_bytes, bytes_to_long
26 import gmpy2 #for precise integer arithmetic
27
28 def send(hsh):#send the message to the server
29     return r.sendline(dumps(hsh)) #send the message to the server
30
31 #Alice's public key
32 alice_N = 222666166575749898681093242521606634709252076906940949533128912823414268805069246485251810142872143501365
33 alice_E = 3 #small public exponent
34
35 #The message we want after decryption
36 message = b'VOTE FOR PEDRO'
answersToQuestions > VOTEFORPEDRO > voteForPedro.py > ...
38 #need to add padding to the message so that when cubed (mod N) it equals the message
39 #3 > N
40 padded_message = b'\x00' * 15 + message
41
42 #moved connection to the server further down
43
44 #converting into integer
45 m = bytes_to_long(padded_message)
46
47 #compute the cube root of m
48 #need to find exact cube root so we need to find k such that k^3 = m
49 #have to use gmpy2 for precise integer arithmetic
50 vote, is_exact = gmpy2.iroot(m, 3)
51
52 # if the vote is not exact, increment it by 1 until we find the perfect cube!!
53 if not is_exact:
54     vote = vote + 1
55
56 print(f"Message: {message}") #the message we want after decryption
57 print(f"Padded message: {padded_message}") #the padded message
58 print(f"Vote value: {vote}") #the vote value
59 print(f"Vote^3 mod N: {pow(vote, 3, alice_N)}") #the cube root of the padded message
60 print(f"Verification: {long_to_bytes(pow(vote, 3, alice_N))}") #the verification
61 print(f"After split: {long_to_bytes(pow(vote, 3, alice_N)).split(b'\x00')[-1]}") #the message after splitting
62
63
64 r = remote("socket.cryptohack.org", 13375) #connection to the server
65 print(r.recv()) #print the message from the server
66
67
68 #####THIS CODE HERE IS FROM THAT GITHUB I REFERENCED, ITS BEEN EDITED BUT THE ORIGINAL CODE IS FROM THAT REPO!#####
69 vote = 85520592299350692515886317752220783
70 option = {
71     'option': 'vote',
72     'vote': hex(vote)
73 }
74 send(option)
75
76 #add the reponse as get instead of get
77 response = loads(r.recv().decode())
78 print(f"Response: {response}")
79
80 if 'flag' in response:
81     print(f"Flag: {response['flag']}")
82 else:
83     print(f"Error: {response['error']}")
84
85 r.close()
86
87 #####

```

10 VOTE FOR PEDRO (150 POINTS)

CATEGORY: RSA

FLAG: CRYPTO{YOUR_VOT3_IS_MY_VOT3}

DIFFICULTY: ADVANCED

LOW EXPONENT ATTACK:

When RSA uses a small public exponent ($e=3$), and the message m is small enough that $m^e < N$, the modular reduction becomes irrelevant. This means: $\text{vote}^3 \bmod N = \text{vote}^3$ (no modular reduction occurs).

This allows us to forge signatures by finding values where the cube produces our desired message after padding removal.

MATHEMATICAL EXPLANATION:

- Normal RSA signing: $\text{signature} = \text{message}^d \bmod N$ (requires private key d)
- Verification: $\text{message} = \text{signature}^e \bmod N$
- With $e=3$ and careful padding, we can find signature values without knowing d

THE PADDING VULNERABILITY:

The server's padding removal (`split(b'\x00')[-1]`) allows us to prepend arbitrary bytes before a null byte separator. This means we can construct `[arbitrary bytes] + \x00 + b'VOTE FOR PEDRO'` When this is the result of vote^3 , the server accepts it as valid.

WHY THIS WORKS:

1. $e=3$ is too small for secure RSA
2. The padding scheme is non-standard and vulnerable
3. The message is short relative to the modulus size
4. We can compute cube roots for small values

VULNERABILITIES DEMONSTRATED:

1. Small public exponents ($e=3$) enable low exponent attacks
2. Improper padding schemes allow signature forgery
3. Short messages with small exponents are particularly vulnerable

PROPER MITIGATION:

1. Use larger public exponents (minimum $e=65537$)
2. Implement PKCS#1 v2.0 (PSS) for signatures
3. Use OAEP padding for encryption
4. Ensure messages are always close to modulus size
5. Follow cryptographic standards and best practices

REAL-WORLD IMPACT:

This type of vulnerability has been found in real systems and demonstrates why:

- Custom cryptographic implementations are dangerous
- Standards like PKCS exist for good reason
- Small optimizations (like $e=3$ for speed) can break security
- Proper padding is critical to RSA security

SUMMARY

TOTAL POINTS:

CHALLENGES OVER VIEW:

Challenge Breakdown:

- Beginner (10-50 pts): 5 challenges (XOR, Base64, Bytes, Encoding, Quadratic)
- Intermediate (60-120 pts): 4 challenges (CRT, Powers, Adrien's, Binomials)
- Advanced (≥ 150 pts): 1 challenge (RSA Decryption, Private Keys, Euler's Totient, Public Keys, Modular Exponentiation)

CATEGORIES COVERED:

- General/Encoding: 4 challenges
- Mathematics: 5 challenges
- RSA: 1 challenge

KEY CONCEPTS DEMONSTRATED:

1. Basic encoding/decoding (Base64, hex, rot13)
2. XOR operations
3. Modular arithmetic
4. Chinese Remainder Theorem
5. Quadratic residues and Legendre symbol
6. RSA vulnerabilities and factorization attacks